
AN0020- EC NB-IoT SDK 编译指南

EigenCOMM Wireless Microcontroller

文档说明

本文档用于指导用户如何对 EC NB-IoT SDK 进行编译，以及如何修改相应的编译文件进行 SDK 的二次开发。

目录

1. 引言	3
1.1 文档目的	3
1.2 主要内容	3
2. 搭建 SDK 编译环境	3
2.1 如何在 windows 下编译	3
2.1.1 安装交叉编译工具	3
2.1.2 编译步骤	4
2.1.3 编译输出	7
2.1.4 分析编译结果	8
3. 使用 SDK 编译环境	9
3.1 SDK makefile 层次结构	10
3.2 Makefile.inc 文件的使用	10
3.3 Makefile 文件的使用	12
3.4 定制化编译的使用	12
3.5 如何创建自己的工程	14
4. 附录	15
4.1 KeilBuild.bat 编译流程举例	15
5. 版本	17
6. 关于移芯通信	错误!未定义书签。

1. 引言

1.1 文档目的

本文档用于帮助用户快速了解 EC NB-IoT SDK 的整体编译和调试环境，并指导用户如何基于此 SDK 增加新的工程进行应用开发。

1.2 主要内容

- 1) 如何搭建 EC NB-IoT SDK 编译环境
- 2) 如何使用 EC NB-IoT SDK 编译示例工程
- 3) 如何创建自己的开发工程

2. 搭建 SDK 编译环境

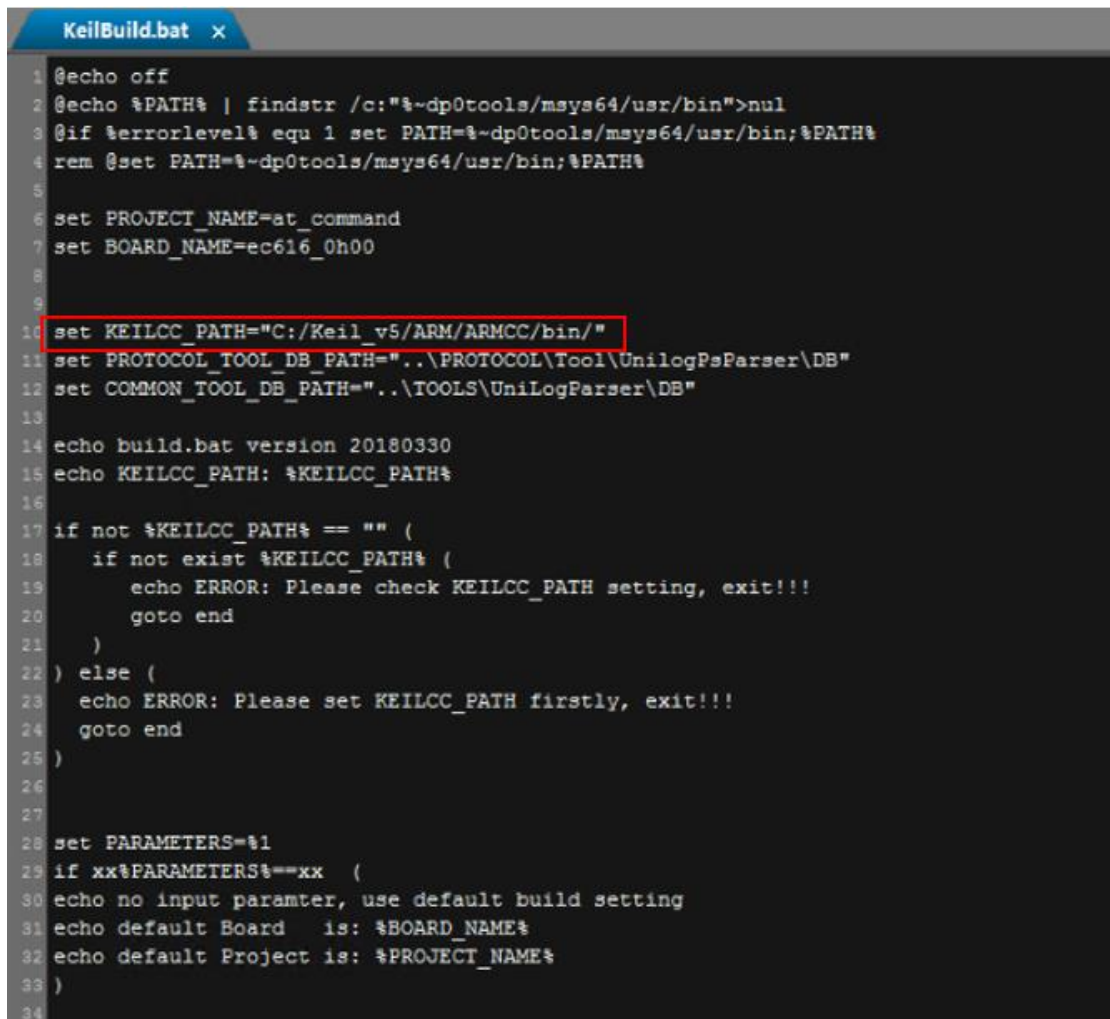
EC NB-IoT SDK 压缩包解压之后，在 SDK 的根目录下会看到支持 EC NB-IoT 编译的几个 windows bat 脚本，例如 KeilBuild.bat 是支持 ARMCC 编译器的 EC NB-IoT 一键编译脚本，而 GccBuild.bat 则是支持 GCC 编译器的 EC NB-IoT 一键编译脚本。EC NB-IoT SDK 实质上可以支持 windows 和 linux 两种编译环境，以及 ARMCC 和 GCC 两种交叉编译方式。本文档目前主要介绍 windows 下 EC NB-IoT SDK 的 ARMCC 的编译，后续会增加其他编译环境的相关介绍。

NOTE: GCC 编译内部开发验证中，暂时不支持。

2.1 如何在 windows 下编译

2.1.1 安装交叉编译工具

对于 ARMCC 编译方式，用户需要安装 keil_v5，安装 keil_v5 的过程会自动安装 ARMCC 交叉编译工具。假设已经安装在 C 盘根目录下，此时更新 ARMCC 交叉编译工具的 PATH 路径到 EC NB-IoT SDK 的一键编译脚本 KeilBuild.bat 中。如下图：



```

1 @echo off
2 @echo %PATH% | findstr /c:"%~dp0tools/msys64/usr/bin">nul
3 @if %errorlevel% equ 1 set PATH=%~dp0tools/msys64/usr/bin;%PATH%
4 rem @set PATH=%~dp0tools/msys64/usr/bin;%PATH%
5
6 set PROJECT_NAME=at_command
7 set BOARD_NAME=ec616_0h00
8
9
10 set KEILCC_PATH="C:/Keil_v5/ARM/ARMCC/bin/"
11 set PROTOCOL_TOOL_DB_PATH="..\PROTOCOL\Tool\UnilogParser\DB"
12 set COMMON_TOOL_DB_PATH="..\TOOLS\UnilogParser\DB"
13
14 echo build.bat version 20180330
15 echo KEILCC_PATH: %KEILCC_PATH%
16
17 if not %KEILCC_PATH% == "" (
18     if not exist %KEILCC_PATH% (
19         echo ERROR: Please check KEILCC_PATH setting, exit!!!
20         goto end
21     )
22 ) else (
23     echo ERROR: Please set KEILCC_PATH firstly, exit!!!
24     goto end
25 )
26
27
28 set PARAMETERS=%1
29 if %PARAMETERS%==%*% (
30     echo no input paramter, use default build setting
31     echo default Board   is: %BOARD_NAME%
32     echo default Project is: %PROJECT_NAME%
33 )
34
  
```

图 1 修改编译器设置

对于 GCC 编译方式，EC NB-IoT SDK 移植了一个精简版本的 msys64 编译工具，用户不需要考虑自行安装，可以直接使用一键编译脚本 GccBuid.bat 进行编译。

NOTE: GCC 编译内部开发验证中，暂时不支持。

2.1.2 编译步骤

第一步：双击 EC NB-IoT SDK 根目录下的 cmd.exe，进入超级终端程序。

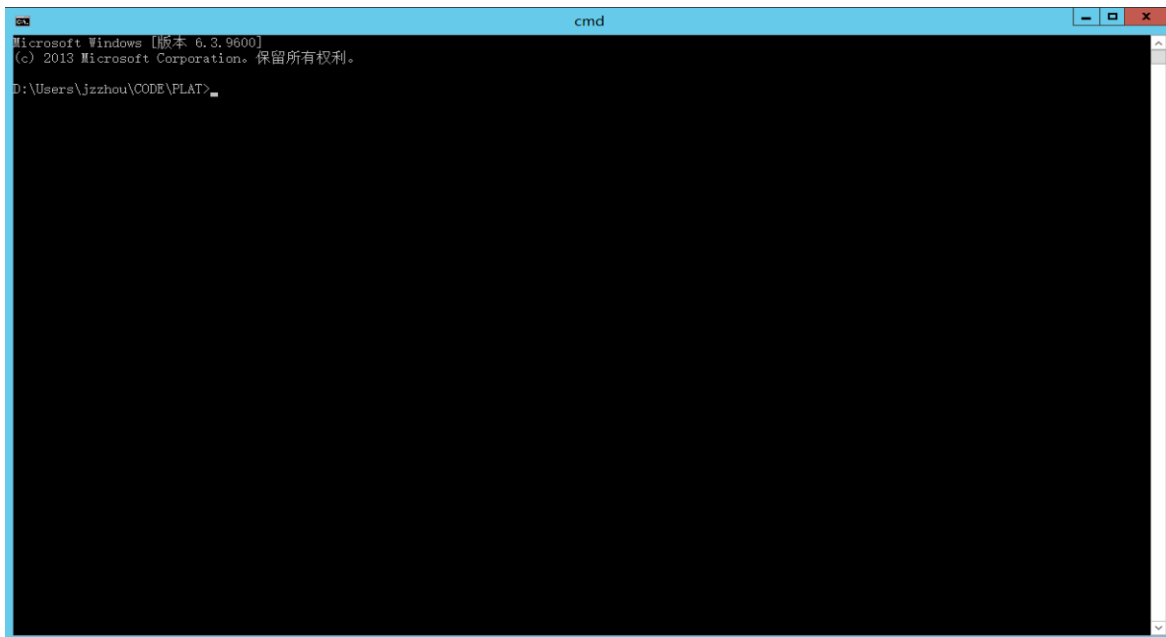


图 2 超级终端程序

第二步：运行 **KeilBuild.bat** 批处理文件，该批处理文件支持通过传入参数来控制不同的编译行为，该参数是可选的，如果用户不指定该传入参数，**KeilBuild.bat** 会按照默认的行为去编译默认板级下的默认工程，否则就会按照参数指定的行为去编译特定的工程。

传入参数的类型及格式为：**#KeilBuild.bat 【BoardName-ProjectName-Option】**

注：BoardName/ ProjectName/ Option 之间使用的是“-”中划线进行连接的

下表以 EC616 为例

支持参数	选项	描述	其他说明
BoardName	ec616_0h00	EC616 板子名称	
ProjectName	at_command	AT 命令 demo 工程	默认编译的工程
	bootloader	Bootloader demo 工程	
	driver_example	驱动 demo 工程	
	fs_example	文件系统 demo 工程	
	onenet_example	移动 onenet demo 工程	
	aliyun_example	阿里云 demo 工程	
	socket_example	Socket demo 工程	
	mqtt_example	Mqtt demo 工程	
	coap_example	Coap demo 工程	
	lwm2m_example	Lwm2m demo 工程	
Option	clean	Clean 当前编译工程	
	clall	Clean 所有工程	
	unilog	编译前预解释生成新的 debug_log.h	
	doc	生成文档	
	verbose	编译过程输出详细的编译命令 log	

	help	输出帮助信息	
	list	列出所有支持的板子和工程	
	allprojects	自动编译所有的 demo 工程	
	defaultall	自动编译默认的工程和 bootloader 工程	

图表 3 所有支持的命令

另外用户也可以通过运行#Keil_build.bat help 来查看所有支持的命令参数以及用法。

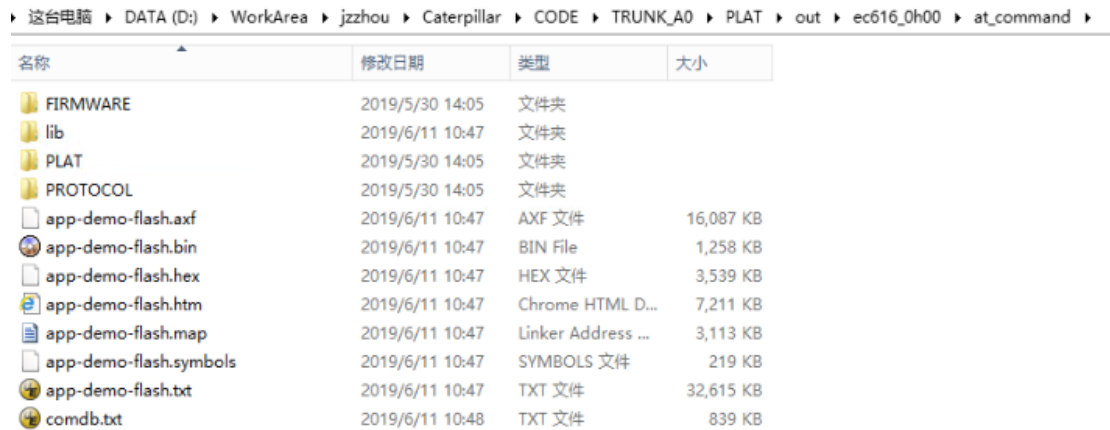


```

cmd
D:\WorkArea\jzzhou\Caterpillar\CODE\TRUNK_A0\PLAT>KeilBuild.bat help
build.bat version 20180330
KEILCC_PATH: "C:/Keil_v5/ARM/ARMCC/bin/"
OPTION: help
=====
-----Show How to Build Project-----
=====
Keilbuild.bat Board-Project-Option
Board,Project,Option could be omitted, if so default Board and project will be used
=====
Supported Options:
doc----run doxygen to generate documents
unilog---pre-parse the log before compile
clean---clean the build output files
clall---clean all output files for every board and project
jenkins_release---trigger jenkins_release
list---list current supported boards and supported project for each board
help---show how to use the build script
=====
Example start -- Example start -- Example start -- Example start
-----
Keilbuild.bat ec616_0h00-driver_example"
build driver_example project for board ec616_0h00"
=====
Keilbuild.bat ec616_0h00-bootloader"
build bootloader project for board ec616_0h00"
=====
Keilbuild.bat ec616_0h00-driver_example-clean"
clean driver_example project for board ec616_0h00"
=====
Keilbuild.bat clean"
clean default project for default board
=====
Keilbuild.bat clall"
clean all output files for every board and project
=====
Keilbuild.bat unilog"
perform log preparse of default project for default board
=====
Keilbuild.bat list"
list current supported boards and supported project for each board

```

图 4 KeilBuild.bat help 选项



名称	修改日期	类型	大小
FIRMWARE	2019/5/30 14:05	文件夹	
lib	2019/6/11 10:47	文件夹	
PLAT	2019/5/30 14:05	文件夹	
PROTOCOL	2019/5/30 14:05	文件夹	
app-demo-flash.axf	2019/6/11 10:47	AXF 文件	16,087 KB
app-demo-flash.bin	2019/6/11 10:47	BIN File	1,258 KB
app-demo-flash.hex	2019/6/11 10:47	HEX 文件	3,539 KB
app-demo-flash.htm	2019/6/11 10:47	Chrome HTML D...	7,211 KB
app-demo-flash.map	2019/6/11 10:47	Linker Address ...	3,113 KB
app-demo-flash.symbols	2019/6/11 10:47	SYMBOLS 文件	219 KB
app-demo-flash.txt	2019/6/11 10:47	TXT 文件	32,615 KB
comdb.txt	2019/6/11 10:48	TXT 文件	839 KB

图 7 输出文件

对于编译过程中的 log 文件，包括错误信息，生成于 SDK/out 目录下，文件名为 outbuildlog.txt:



名称	修改日期	类型	大小
ec616_0h00	2019/5/30 16:13	文件夹	
outbuildlog.txt	2019/6/11 10:48	TXT 文件	69 KB

图 8 编译 log 文件

2.1.4 分析编译结果

对于 ARMCC 编译方式，用户可以通过 $\$(BUILDDIR)/\$(BINNAME).map$ 文件来查看生成的映像文件信息图，该映像文件信息图包括映像文件中的每个加载域，运行域和输入段的大小和地址，也包含了整个映像文件的 RAM/FLASH 使用统计信息，另外也可以查看每个 LIB 库的 RAM/FLASH 占用情况，用户可以基于此信息，进行有针对性的代码优化。

900918	206492	85472	13132	200796	5061684	Library Totals
468	0	205	42	5	0	(incl. Padding)

Code (inc. data)	RO Data	RW Data	ZI Data	Debug	Library Name	
7526	742	696	45	804	223214	libdriver.lib
23632	4948	777	140	66008	276210	libfreertos.lib
111508	8832	20555	1250	66636	1138577	libfw.lib
76360	13590	2586	239	18752	529492	libmiddleware_3dparty.lib
48718	8242	1815	2451	27889	408551	libmiddleware_ec.lib
629836	170020	58705	8953	20702	2479494	libps.lib
132	36	0	0	0	3154	libstartup.lib
2248	82	133	12	0	2568	mc_w.l
490	0	0	0	0	424	mf_w.l

900918	206492	85472	13132	200796	5061684	Library Totals

=====						
Code (inc. data)	RO Data	RW Data	ZI Data	Debug		
902288	206578	85552	13132	201820	4896687	Grand Totals
902288	206578	85552	13132	201820	4896687	ELF Image Totals
902288	206578	85552	13132	0	0	ROM Totals
=====						
Total RO	Size (Code + RO Data)			987840	(964.69kB)	
Total RW	Size (RW Data + ZI Data)			214952	(209.91kB)	
Total ROM	Size (Code + RO Data + RW Data)			1000972	(977.51kB)	
=====						

图 9 MAP 文件分析

3. 使用 SDK 编译环境

本章节将进一步介绍 EC NB-IoT SDK 的编译细节，指导用户如何使用 SDK 的编译规则，在增加不同级别的代码时如何修改相应的 Makefile 文件来满足不同的编译需求。

3.1 SDK makefile 层次结构

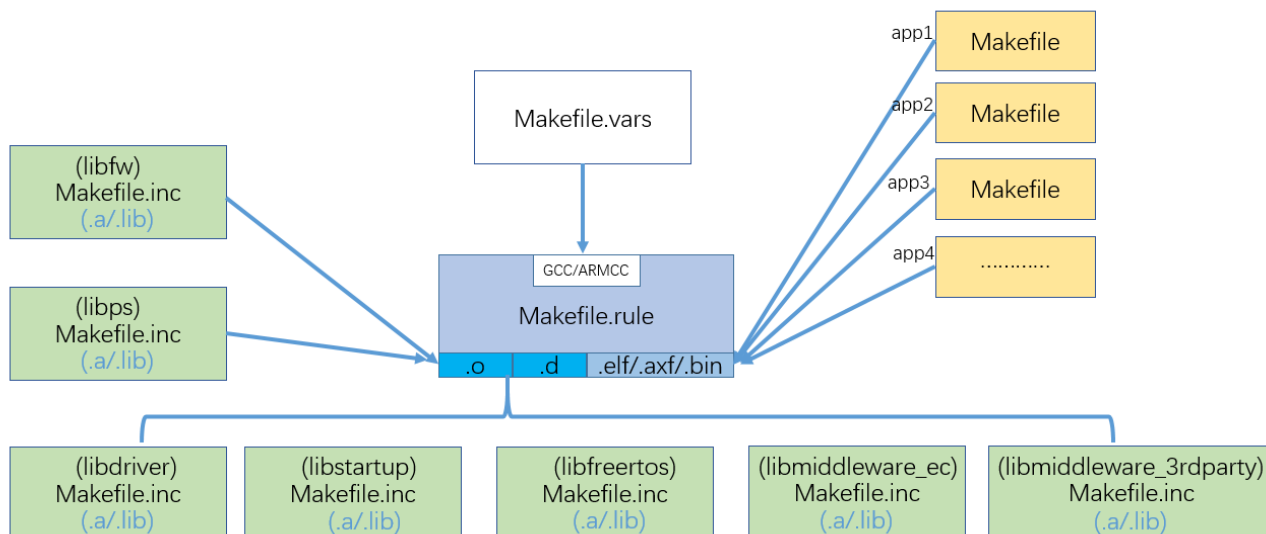


图 10 编译总体结构

如图 10 所示，**Makefile.rule** 文件是 EC NB-IoT SDK 编译组件的核心，同时包含了对 GCC 和 ARMCC 这两种编译方式的支持，实现了通用.o 目标文件以及依赖文件.d 的生成规则，同时支持增量编译。**Makefile.var** 文件抽象了 **Makefile.rule** 所使用的一些编译变量，不同的 **Makefile.inc** 分布在不同的待编译目录中，将 SDK 目录下的所有文件夹串联起来。为了生成最终的目标文件，需要通过应用层面的 **Makefile** 来指明需要生成的目标 bin 文件来实现。

3.2 Makefile.inc 文件的使用

本章节以 SDK\middleware\thirdparty\wakaama 下的 **Makefile.inc** 为例，说明如何书写 **Makefile.inc** 来编译一个新的模块，用户在增加新的编译模块时可以参考此文件来改写。

//源文件所在的根路径

LWM2M_DIR := \$(TOP)/PLAT/middleware/thirdparty/wakaama

// 该模块需要依赖的 CFLAGS 编译选项

CFLAGS += -DLWM2M_OLD_CONTENT_FORMAT_SUPPORT -DNO_DECLTYPE -DLWM2M_BOOTSTRAP -DWITH_LOGS -DLWM2M_WITH_LOGS

// 该模块需要依赖的头文件编译选项

CFLAGS_INC += -I \$(LWM2M_DIR)/core \
 -I \$(LWM2M_DIR)/core/er-coap-13 \
 -I \$(LWM2M_DIR)/examples/client \
 -I \$(LWM2M_DIR)/examples/shared

// 该模块参与编译的所有源文件路径

LWM2M_SRC_DIRS += \$(LWM2M_DIR)/core \
 \$(LWM2M_DIR)/core/er-coap-13 \
 \$(LWM2M_DIR)/examples/client \
 \$(LWM2M_DIR)/examples/shared

// 需要排除在外的编译文件

```
LWM2M_EXCLUDE_FILES :=
// 扫描所有源文件
LWM2M_CSRC = $(foreach dir, $(LWM2M_SRC_DIRS), $(wildcard $(dir)/*.c))
// 剔除不需要编译的文件
LWM2M_CFILES = $(filter-out $(LWM2M_EXCLUDE_FILES), $(LWM2M_CSRC))
// .c 替换成.o
LWM2M_COBJSTEMP := $(patsubst %.c, %.o, $(LWM2M_CFILES))
// 绝对路径转化为相对路径
LWM2M_COBJSTEMP := $(subst $(LWM2M_DIR), PLAT/middleware/thirdparty/wakaama, $(LWM2M_COBJSTEMP))
// 前缀切换到$(BUILDDIR)路径
LWM2M_COBJS := $(addprefix $(BUILDDIR)/, $(LWM2M_COBJSTEMP))

// 包含依赖文件生成
#include $(LWM2M_COBJS:.o=.d)

// GCC 编译分支
ifeq ($(TOOLCHAIN), GCC)
// GCC 特殊头文件包含
CFLAGS_INC += -I $(TOP)/PLAT/os/freertos/portable/gcc
// 声明生成.a lib 库
lib-y += liblwm2m.a
// GCC 库的编译规则
$(BUILDDIR)/lib/liblwm2m.a: $(LWM2M_COBJS)
    @mkdir -p $(dir $@)
    $(ECHO) AR $@
    $(Q)$ (AR) -cr $@ $^
endif

// ARMCC 编译分支
ifeq ($(TOOLCHAIN), ARMCC)
// ARMCC 特殊头文件包含
CFLAGS_INC += -I $(TOP)/PLAT/os/freertos/portable/keil
// 声明生成.lib 库文件
lib-y += liblwm2m.lib
// ARMCC 库的编译规则
$(BUILDDIR)/lib/liblwm2m.lib: $(LWM2M_COBJS)
    @mkdir -p $(dir $@)
    $(ECHO) AR $@
    $(Q)$ (AR) $(ARFLAGS) $@ $^
endif

endif
```

3.3 Makefile 文件的使用

本章节以 SDK\project\ec616_0h00\apps\onenet_example\ARMCC 下的 Makefile 为例，说明如何书写 Makefile 来编译一个新的工程，用户在增加自己的开发工程时可以参考使用。

```
// 声明该工程支持的板级目标
AVAILABLE_TARGETS = ec616_0h00
// 指明编译选择的 toolchain
TOOLCHAIN          = ARMCC
// 定义生成 BIN 的文件名
BINNAME             = onenet-demo-flash
// 定义 TOP 目录
TOP                 := ../../../../..
// 定制化编译选项
BUILD_AT            = n
BUILD_AT_DEBUG      = n
THIRDPARTY_CISONENET_ENABLE = y
THIRDPARTY_WAKAAMA_ENABLE = n
// 需要增加的头文件路径
CFLAGS_INC          += -I ../inc
// 需要增加的源文件编译
obj-y               += PLAT/project/$(TARGET)/apps/onenet_example/src/app.o \
                       PLAT/project/$(TARGET)/apps/onenet_example/src/cisAsynEntry.o \
                       PLAT/project/$(TARGET)/apps/onenet_example/src/bsp_custom.o

// include Makefile.rules 编译规则
include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

3.4 定制化编译的使用

EC NB-IoT SDK 中包含了很多自研和第三方开源的软件，通过将不同功能的软件进行模块化处理，减少相互之间的编译耦合，从而方便用户根据需求进行高度颗粒化的定制编译，从而大大提高代码的编译效率。高度颗粒化的定制特性体现在对 EC NB-IoT 的所有外设驱动，第三方开源软件以及协议栈软件，SDK 都支持对其进行定制化编译。

EC NB-IoT SDK 根据不同的应用场景，整理了一套默认的模式定制化编译配置，用户如果在自己工程的 Makefile 文件中没有对定制化编译选项进行新的定制，编译系统就会根据这套默认的模式定制化编译配置进行编译。用户可以通过 SDK \device\eigencomm\board\ecxxx_0h00 目录下的 ecxxx_0h00.mk 来查看 EC NB-IoT SDK 所有支持的定制化编译选项。

对于是否支持 AT 命令，来选择是编译双芯片方案还是 OpenCPU 方案，主要是通过配置下面几个选项来实现的：

```
BUILD_AT            ?= y
BUILD_AT_DEBUG      ?= y
```

对于类似 bootloader 工程的编译，不需要开启 OS 操作系统代码的编译，可以通过下面这个选项来控制：

```
BUILD_OS            ?= y
```

对于外设驱动来说，通常使能编译的概率比较大，所以默认编译配置里都是使能编译的。如下：

<code>DRIVER_USART_ENABLE</code>	<code>?= y</code>
<code>DRIVER_SPI_ENABLE</code>	<code>?= y</code>
<code>DRIVER_I2C_ENABLE</code>	<code>?= y</code>
<code>DRIVER_DMA_ENABLE</code>	<code>?= y</code>
<code>DRIVER_CACHE_ENABLE</code>	<code>?= y</code>
<code>DRIVER_PMU_ENABLE</code>	<code>?= y</code>
<code>DRIVER_FLASHRT_ENABLE</code>	<code>?= y</code>
<code>DRIVER_FLASH_ENABLE</code>	<code>?= y</code>
<code>DRIVER_GPIO_ENABLE</code>	<code>?= y</code>
<code>DRIVER_GPR_ENABLE</code>	<code>?= y</code>
<code>DRIVER_TIMER_ENABLE</code>	<code>?= y</code>
<code>DRIVER_UNILOG_ENABLE</code>	<code>?= y</code>
<code>DRIVER_WDG_ENABLE</code>	<code>?= y</code>
<code>DRIVER_IC_ENABLE</code>	<code>?= y</code>
<code>DRIVER_CIPHER_ENABLE</code>	<code>?= y</code>
<code>DRIVER_CIPHER_FLH_ENABLE</code>	<code>?= y</code>
<code>DRIVER_EFUSE_ENABLE</code>	<code>?= y</code>
<code>DRIVER_CIPHER_TLS_ENABLE</code>	<code>?= y</code>
<code>DRIVER_MCUMODE_ENABLE</code>	<code>?= y</code>
<code>DRIVER_ADC_ENABLE</code>	<code>?= y</code>

如果用户在评估应用需求的时候，觉得某个驱动模块是不需要使用的，可以在自己工程的 **Makefile** 里将该模块的编译选项置为 **n**。例如：用户评估下来不需要使用 **SPI** 和 **I2C** 外设，可以如下修改：

```
AVAILABLE_TARGETS = ec616_0h00 ec616z0_0h00
TOOLCHAIN          = ARMCC
BINNAME            = lwm2m-demo-flash
TOP                := ../../../../../../..

BUILD_AT           = n
BUILD_AT_DEBUG     = n
THIRDPARTY_CISONENET_ENABLE = n
THIRDPARTY_SWING_ENABLE   = n
THIRDPARTY_WAKAAMA_ENABLE = y
THIRDPARTY_MBEDTLS_ENABLE = n
THIRDPARTY_TINYDTLS_ENABLE = y
DRIVER_SPI_ENABLE   = n
DRIVER_I2C_ENABLE   = n

CFLAGS_INC += -I ../inc
obj-y      += PLAT/project/$(TARGET)/apps/lwm2m_example/src/app.o \
              PLAT/project/$(TARGET)/apps/lwm2m_example/src/bsp_custom.o
include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

图 11 驱动定制化编译配置

注：定制化默认配置选项使用的是“**?**”，表示如果 **Makefile** 中没有定义该编译选项，就会使用该默认的配置。相反，工程中 **Makefile** 文件里使用“**=**”，表示覆盖替换掉默认配置，启用新定义的配置。

同样，对于 **Middleware** 中的自研模块和第三方开源模块，**EC NB-IoT SDK** 也定义了对应的定制化编译选

项，有所不同的，对于这类模块，通常禁止编译的概率比较大，所以默认的编译配置里大部分都是禁止编译的，但也有例外，例如 `littlefs`, `lwip`, `ping`, `rohc` 等模块对于 NB 设备来说是必须的，所以这些模块默认也是使能编译的。

#Choose if below thirdparty software need to be compiled

```
THIRDPARTY_CISONENET_ENABLE    ?= n
THIRDPARTY_WAKAAMA_ENABLE      ?= n
THIRDPARTY_CTWING1_5_ENABLE     ?= n
THIRDPARTY_MQTT_ENABLE          ?= n
THIRDPARTY_LIBCOAP_ENABLE       ?= n
THIRDPARTY_HTTPC_ENABLE         ?= n
THIRDPARTY_LITTEFS_ENABLE       ?= y
THIRDPARTY_IPERF_ENABLE         ?= y
THIRDPARTY_CJSON_ENABLE         ?= n
THIRDPARTY_LWIP_ENABLE          ?= y
THIRDPARTY_PING_ENABLE          ?= y
THIRDPARTY_ROHC_ENABLE          ?= y
THIRDPARTY_MBEDTLS_ENABLE       ?= n
THIRDPARTY_RTT_ENABLE           ?= y
THIRDPARTY_ALIIOT_ENABLE        ?= n
```

3.5 如何创建自己的工程

首先拷贝 SDK\project\$(TARGET) \templates\app_demo 模板工程到 SDK\project\$(TARGET) \apps 目录下，然后在该目录下：

- (1) 修改文件夹名称 `app_demo` 为新的工程名称 `xxxxx`。
- (2) 在 `src` 和 `inc` 目录中添加源码文件和头文件实现新的应用
- (3) 修改 ARMCC 下的 `Makefile` 目标文件名称和需要编译的原文件

```
AVAILABLE_TARGETS = ecxxx_0h00
TOOLCHAIN = ARMCC
TOP := ../../../../..
// 配置新的定制化编译选项
BUILD_AT = n
BUILD_AT_DEBUG = n
THIRDPARTY_CISONENET_ENABLE = n
THIRDPARTY_CTWING1_5_ENABLE = n
THIRDPARTY_WAKAAMA_ENABLE = y
// 定义新的目标文件名称
BINNAME = xxx-demo-flash
// 增加新的编译文件
```



```
obj-y += sdk/project/$(TARGET) / apps/src/app.o \
        sdk/project/$(TARGET) / apps/src/bsp_custom.o \
        sdk/project/$(TARGET) / apps/src/xxxx.o
include $(TOP)/sdk/tools/scripts/Makefile.rules
```

- (4) 修改 SDK 根目录下的一键编译脚本 KeilBuild.bat 添加新的工程文件配置，以增加新的工程 lwm2m_example 为例，如下图：

```
echo PROJECT_NAME: %PARAMETERS% | findstr "sleep_example"
if not errorlevel 1 (
    set PROJECT_NAME=sleep_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "flash_example"
if not errorlevel 1 (
    set PROJECT_NAME=flash_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "fs_example"
if not errorlevel 1 (
    set PROJECT_NAME=fs_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "onenet_example"
if not errorlevel 1 (
    set PROJECT_NAME=onenet_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "socket_example"
if not errorlevel 1 (
    set PROJECT_NAME=socket_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "https_example"
if not errorlevel 1 (
    set PROJECT_NAME=https_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "lwm2m_example"
if not errorlevel 1 (
    set PROJECT_NAME=lwm2m_example
)

echo PROJECT_NAME: %PARAMETERS% | findstr "mqtt_example"
if not errorlevel 1 (
    set PROJECT_NAME=mqtt_example
)
```

图 12 增加新的工程配置

4. 附录

4.1 KeilBuild.bat 编译流程举例

- (1) KeilBuild.bat list

```
D:\WorkArea\yjzhong\workA0\PLAT>KeilBuild.bat list
build.bat version 20180330
KEILCC_PATH: "C:/Keil_v5/ARM/ARMCC/bin/"
OPTION: list
-----Supported Board-----
ec616_0h00
ec617_0h00
-----
*****supported projects for board: ec616_0h00*****
----abupfota_example
----at_command
----bootloader
----coap_example
----ct_example
----driver_example
----flash_example
----fs_example
----fs_example_inter
----https_example
----lwm2m_example
----mqtt_example
----multi_at_command
```

图 13 支持 Option 列表

(2) KeilBuild.bat lwm2m_example-unilog

由于 Option 带 unilog 的话编译之前会花大量时间去解释生成新的 unilog 数据库文件，所以用户可以先只运行 KeilBuild.bat lwm2m_example，如果提示 unilog 没有定义的错误时再去运行一次带 unilog 的编译。类似 unilog 没有定义的错误如下：

```
D:\WorkArea\jzzhou\Caterpillar\CODE\TRUNK_A0\PLAT>KeilBuild.bat
build.bat version 20180330
KEILCC_PATH: "C:/Keil_v5/ARM/ARMCC/bin/"
no input paramter, use default build setting
default Board is: ec616_0h00
default Project is: at_command
Start time: 2019/06/11 周二 13:35:11.06
(cd project/ec616_0h00/apps/at_command/ARMCC && make V=0 -f Makefile all)
make[1]: Entering directory '/cygdrive/d/WorkArea/jzzhou/Caterpillar/CODE/TRUNK_A0/PLAT/project/ec616_0h00/apps/at_command/ARMCC'
CC ../../../../../../PLAT/project/ec616_0h00/apps/at_command/src/app.c
../../../../../../../../PLAT/project/ec616_0h00/apps/at_command/src/app.c", line 59: Error: #20: identifier "UNILog_PLA_APP_at_recv_eve
COMM_TRACE(UNILog_PLA_APP, at_recv_event_4, P_DEBUG, 1, "at cmd recv err evt=0x%x", event);
../../../../../../../../PLAT/project/ec616_0h00/apps/at_command/src/app.c: 0 warnings, 1 error
make[1]: *** [../../../../../../../../PLAT/tools/scripts/Makefile.rules:152: ../../../../../../PLAT/out/ec616_0h00/at_command/PLAT/projec
t_command/src/app.o] Error 1
make[1]: Leaving directory '/cygdrive/d/WorkArea/jzzhou/Caterpillar/CODE/TRUNK_A0/PLAT/project/ec616_0h00/apps/at_command/ARMCC'
make: *** [Makefile:11: keilall] Error 2
fail
#####
##
##          #####      ##      ##      ##      ##
##          ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##
##          #####      ##      ##      ##      ##
##          ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##
##          ##      ##      ##      ##      ##
##          #####      ##      ##      ##      ##
#####
D:\WorkArea\jzzhou\Caterpillar\CODE\TRUNK_A0\PLAT>_
kefile:11: keilall] Error 2WorkArea/jzzhou/Caterpillar/CODE/TRUNK_A0/PLAT/project/ec616
```

图 14 unilog 没有定义的错误

5. 版本

版本	日期	备注
Draft	2018-11-11	
A	2018-12-01	

6. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com